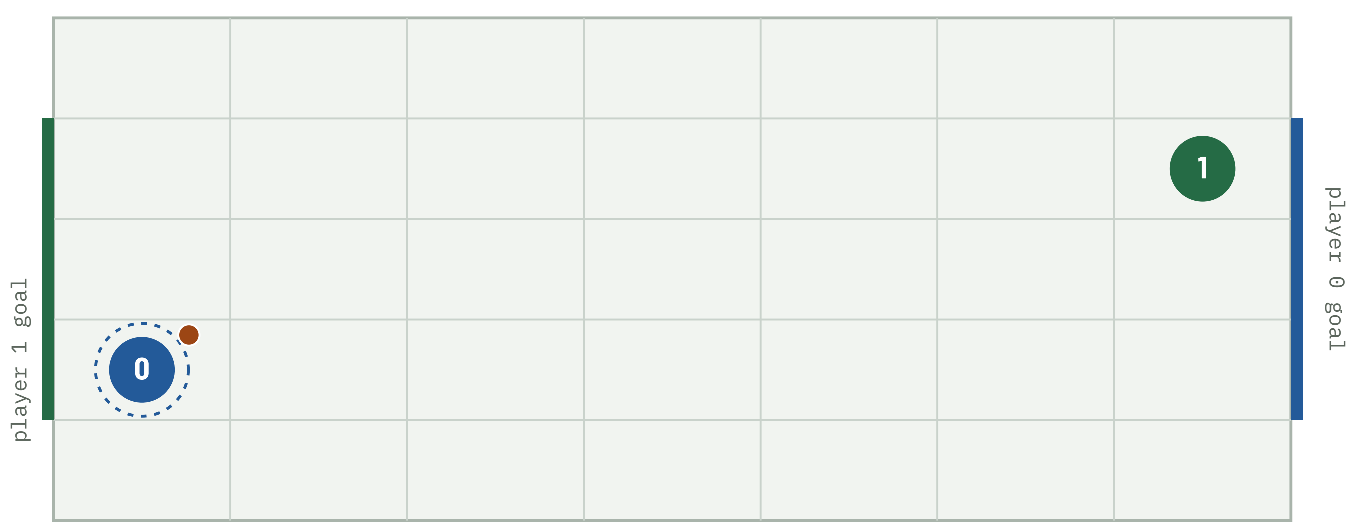
MULTI-AGENT REINFORCEMENT LEARNING · RESEARCH REPORT

When Does Soccer Need Mixed Strategies?

A pure-first Nash-Q approach: where a two-player soccer Markov game does — and does not — need mixing, and how much LP work that saves.

> FOUR QUESTIONS
>
> **RQ1 Existence** — under what transition structures does a pure equilibrium exist at every state? **RQ2 Efficiency** — how much work does checking for a pure saddle first eliminate? **RQ3 Mechanism** — which spatial configurations force mixing? **RQ4 Robustness** — how do discounting and tolerance affect the split?

repo · soccer-markov-nash | 210+ tests | game family in `docs/design.md` | methods in `docs/methods.md` | every policy drawn in `docs/gallery.html`



state = (x0, y0, x1, y1, b) = (0, 1, 6, 3, 0)

The kickoff. 7×5 grid, 2 380 non-terminal states. Player 0 (blue) carries the ball and attacks the right edge; player 1 (green) the left. Start positions are the ones the A10 page assigns this netID. Actions `U D L R` are chosen simultaneously; reward is `+1 / -1 / 0`, zero-sum.

ABSTRACT

1 **With deterministic move resolution — solved exactly, undiscounted, by 100-step backward induction — every one of the 238 000 (state × step) stage games has a pure saddle.** A pure equilibrium exists; the LP is never called; discounting was never load-bearing. It is also positionally determined, with an explicit pure memoryless profile.

2 **A coin-flip tie-break does not change this;** Littman's random *move order* does — but only when it is the majority rule (a sharp threshold at `blend = 0.5`), only when the goal mouth exceeds one cell, and even then on 3.95% of states (7×5, 3-cell goal).

3 **Those mixed states are geometrically localised** (a decision tree predicts them at precision 0.96) and **structurally uniform**: 68 of 94 have a 2×2 matching-pennies support. The region is invariant to the kickoff and to the reward objective — it depends only on the resolution rule and the goal width.

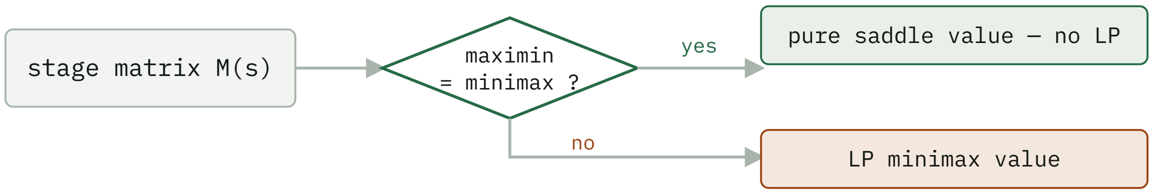
4 **The pure-first hybrid solver** matches the all-LP value function to `4e−16` while calling the LP on only 3.95% of states — ~25× fewer per sweep — though those states carry 41% of the equilibrium-path occupancy; mirror symmetry halves the work again.

Method

The game is zero-sum, so at every state the stage game is a payoff matrix $M(s)[a_0, a_1] = E[R + \gamma V(s')]$ and the Shapley (1953) fixed point is the minimax value:

$$
\begin{aligned}
V(s) = \mathrm{val}(M(s)) &= \max_{n} \min_{k3} (p^{\top}M)[a_1] \\
&= \min_{p} \max_{a0} (Mq)[a_0]
\end{aligned}
$$

The $Q = R + \beta\,\mathrm{Nash}(Q')$ update is this operator. The A10 game is *undiscounted* (±1 at a goal, tie after 100 steps); $\gamma < 1$ is a contraction device for value iteration and RQ4 shows the split holds across γ. Three stage backups: **pure** (the maximin security value — a fast lower bound, not a Nash value when it is below the minimax), **mixed** (the LP minimax value, via `scipy` HiGHS), and **hybrid** (the pure saddle value where `maximin = minimax`, the LP elsewhere — exact everywhere). Support enumeration (`support_enum.py`) finds *all* equilibria, including of general-sum games.



The pure-first hybrid. On the deterministic game the "yes" branch is taken at every one of the 2 380 states, so the LP is never called.

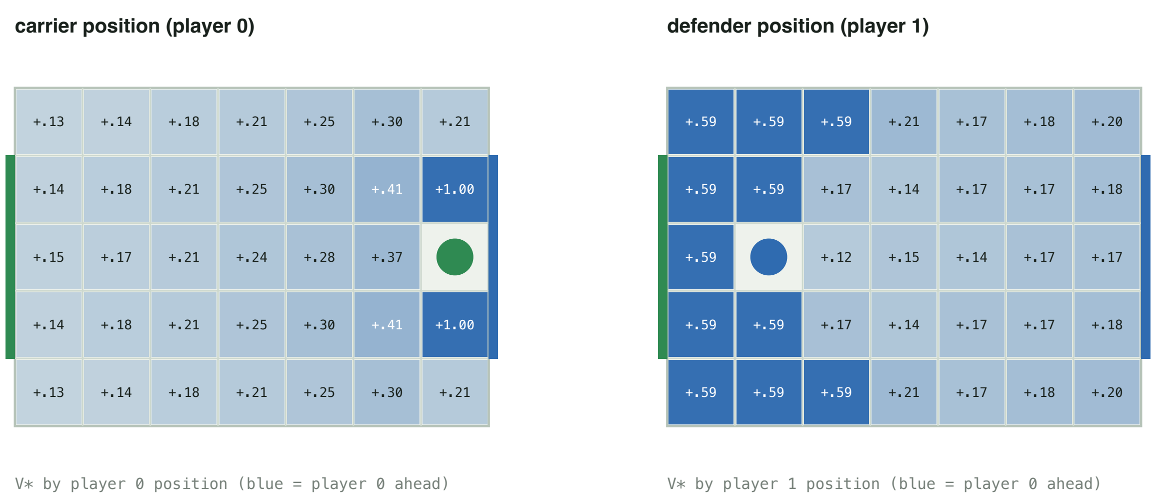
RQ1 — Existence

The A10 game is undiscounted, so it is solved exactly by 100-step backward induction (`run_finite_horizon`, `experiments/undiscounted.csv`):

EXACT UNDISCOUNTED GAME — 238 000 (STATE × STEP) STAGE GAMES

MOVE ORDER	MIXED STAGE GAMES	V(KICKOFF)	PURE EQUILIBRIUM?
deterministic (exact A10)	0	0.000	yes — pure (non-stationary)
coinflip tie-break	0	0.000	yes
random move order	3 148 (404 states)	+0.459	no

The deterministic equilibrium is **constructive and memoryless**: each player's win attractor (states from which it forces a goal, `soccer_nash/attractor.py`) equals its `V* = ±1` set, and the pure profile "attractor move on your winning set, safety move elsewhere" realizes `V*` at every state (`scripts/positional.py`) — the game is positionally determined.



The value surface of the random-order game (γ = 0.9). Left: V* by carrier position — a smooth gradient toward the attacking goal, saturating to +1 on the two cells that score in one move. Right: V* by defender position with the carrier held near midfield — nearly flat, the defender's location barely moves the value.

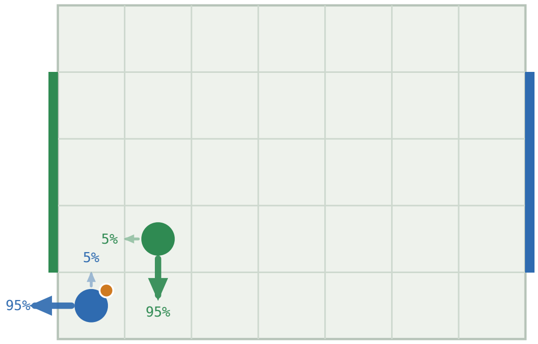
The stationary $\gamma < 1$ solve agrees and is faster — 0 no-pure-saddle stage games on the deterministic game at every γ from 0.5 to 0.995, and **94** / **2380** on the random game at `γ = 0.9` (`V(kickoff) = +0.164` at the netID start). The no-pure-saddle count never depends on the kickoff.

IT IS THE MOVE ORDER, NOT THE RANDOMNESS — AND IT HAS TO BE THE MAJORITY

The deterministic and coin-flip value functions are *identical* (`max |V_det − V_coin| = 0` at every γ): optimal play never enters a losing contest, so the coin is never flipped. Only Littman's random move order — where a ball-steal depends on who resolves first *and* on both players' targets — creates matching-pennies subgames. Interpolating the two (`move_order="blend"`, resolve by random order with probability `blend`) shows the onset is **sharp**: **0** mixed stage games for `blend ≤ 0.5`, dozens at `blend = 0.52`, and an *overshoot* to ~1.5× the fully-random count near `blend ≈ 0.7` before it relaxes (`scripts/blend.py`, `docs/blend.md`). A minority of random-order resolution is washed out by the deterministic tie-break.

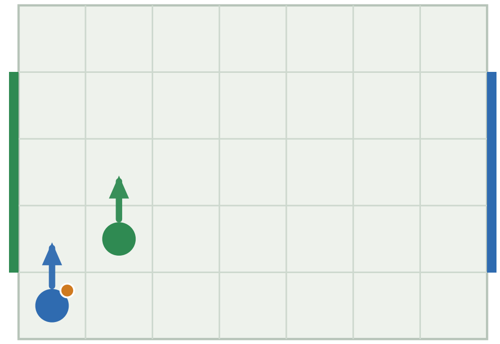
A threshold, not a ramp. No-pure-saddle stage games vs. the fraction of moves resolved by random order (5×4 board). Nothing until the random rule is the majority; then a jump and an overshoot.

random order -> mixed



mixed equilibrium · V = +0.078

deterministic -> pure



pure equilibrium · V = +0.000

One state, two resolution rules. Arrows are the equilibrium action distribution, sized by probability. Random order (Littman) makes the stage game matching pennies; deterministic carrier-wins collapses it to a pure best reply. Full set: docs/gallery.html.

RQ2 — Efficiency

Two numbers, kept apart: the **LP-call rate** is a property of the game and reproduces exactly; **wall clock** is a median of 5 repeats and depends on the machine and LP backend (`scripts/benchmark.py`).

RANDOM MOVE ORDER, Γ = 0.9 — THE CASE WHERE THE LP IS ACTUALLY NEEDED

SOLVER	SWEEPS	LP CALLS	STATES NEEDING LP	MEDIAN WALL CLOCK
pure (lower bound)	18	0	—	0.3 s
hybrid exact	108	9 672	3.95%	13.2 s
all-LP exact	108	~3.6 M	100%	~8 min

The hybrid solves an LP on only the **3.95%** of states lacking a pure saddle — a ~25× per-sweep reduction — and matches the all-LP value to `4e-16`. The wall-clock ratio (~13 s vs ~8 min) is larger but less portable: it also reflects matrix construction and the LP backend. **Mirror symmetry** (`run_symmetric`, deterministic dynamics only): every state has a distinct board-flip-plus-player-swap mirror, so the 2 380 states are 1 190 pairs — reconstruct one from the other by `V(mirror(s)) = -V(s)`, cutting the deterministic solve 0.33 s → 0.23 s.

3.95% OF STATES, BUT 41% OF THE EQUILIBRIUM PATH

The LP-call rate counts the whole state space. Under the Nash policy from the kickoff only **456 of 2 380** states are ever reached, and the 94 no-pure-saddle states carry **41%** of the discounted occupancy — a 2× over-representation on the path, 10× vs. the raw count (`soccer_nash/occupancy.py`, `docs/occupancy.md`). The carrier's optimal march to goal runs straight through the contested band; four of the eight most-visited states are mixed. The mixed structure is rare to *store* and common to *use*.

Freeze-then-iterate, and its staleness

RANDOM MOVE ORDER, HYBRID SOLVER — SAME FIXED POINT, |DIFF| < 3E-9

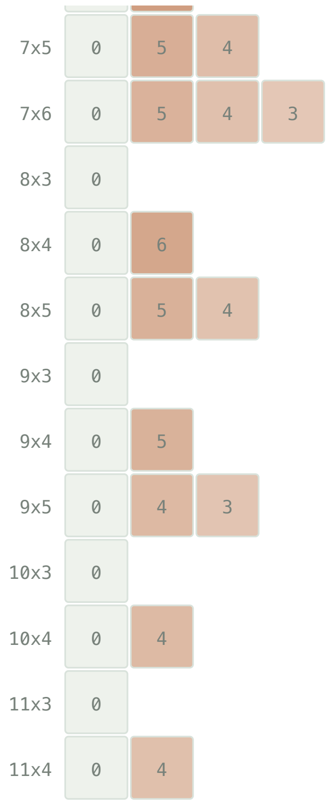
	ROUNDS / SWEEPS	MATRIX-GAME SOLVES	WALL CLOCK
value iteration	108	9 672	13.8 s
policy iteration	21	1 879	17.2 s

5× fewer LP solves — but the frozen strategies' distance to the current Nash stays *maximal* (a full pure-strategy flip) for **16 outer rounds**, then collapses to `< 1e-8`. The thrashing eats the saving; on the deterministic game it is strictly worse. Value iteration with the per-sweep Nash cache wins here.

RQ3 — Mechanism

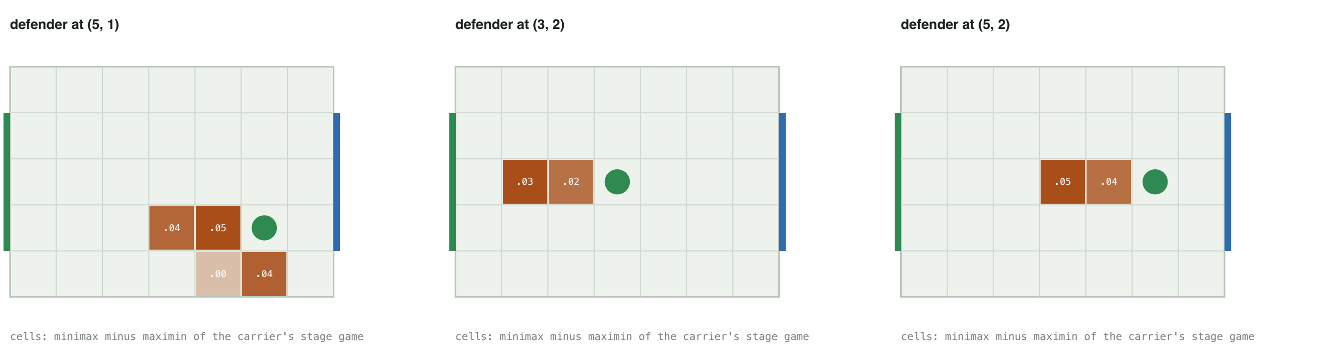
Goal-mouth width, not board size, is the gate. The phase diagram sweeps every board with `width ≤ 11`, `height ≤ 9`, `width·height ≤ 45` against goal widths 1 to `height−2` (`scripts/phase_diagram.py --analyze`, `experiments/phase_diagram.csv`).

	goal–mouth width						
	1	2	3	4	5	6	7
3x3	0						
3x4	0	12					
3x5	0	10	9				
3x6	0	7	7	7			
3x7	0	7	6	6	6		
3x8	0	6	6	5	5	5	
3x9	0	6	5	5	4	4	4
4x3	0						
4x4	0	8					
4x5	0	8	6				
4x6	0	7	6	5			
4x7	0	7	5	5	4		
4x8	0	6	5	5	4	4	
4x9	0	6	5	4	4	4	3
5x3	0						
5x4	0	7					
5x5	0	6	4				
5x6	0	6	4	4			
5x7	0	5	4	4	3		
5x8	0	5	4	3	3	3	
5x9	0	5	4	3	3	3	3
6x3	0						
6x4	0	8					
6x5	0	6	4				
6x6	0	5	4	3			
6x7	0	4	4	4	3		
7x3	0						
7x4	0	7					



cells = mixed-state %

The gate. Goal width 1 → **0** mixed states, on all 40 one-cell configs. Goal width ≥ 2 → mixing on all 87: whether a board mixes is a *perfect* function of goal width. The *fraction* (2.7–12%) is an OLS R^2 0.64 in board area — a dilution effect — with goal width adding little and a negative coefficient. Every state is reachable from the kickoff.



Where the carrier has to guess. Defender pinned at the green disc; each cell shaded by `minimax − maximin` of the carrier's stage game (0 = pure saddle). Mixing is a local phenomenon — a handful of cells where the defender contests the forward lane.

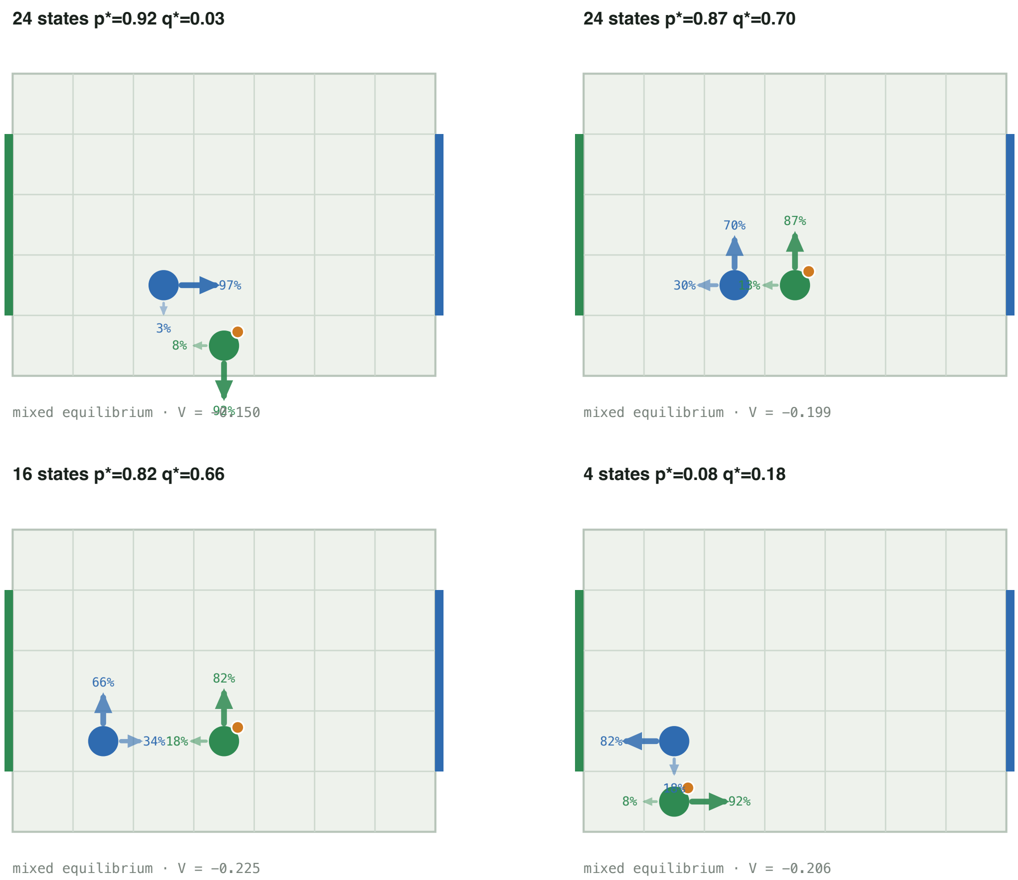
Geometry predicts the label (`scripts/geometry_model.py`): a depth-4 decision tree on carrier-frame features separates `mixed` from the rest with **precision 0.96, recall 0.91**. The dominant feature is `defender_can_intercept` — the defender within one move of the carrier's forward cell (`P(mixed) = 0.44` vs `0.002`).

Beyond prediction, the 94 mixed states canonicalize under the board mirror to 47 pairs and cluster into **8 geometric templates** (`scripts/templates.py`, `docs/templates.md`).

THE MIXED-STATE TEMPLATES — CARRIER-FRAME GEOMETRY, EQUILIBRIUM SUPPORT

STATES	GEOMETRY	SUPPORT	STRUCTURE
24	defender 1 ahead, carrier one row off the goal row	2×2	matching pennies
24	defender 1 ahead, same row, carrier on a goal row	2×2	matching pennies
16	defender 2 ahead, same row, carrier on a goal row	2×2	matching pennies
4	defender diagonally adjacent, carrier off the goal row	2×2	matching pennies
14	defender 1–2 ahead, same row, on a goal row	2×1	near-pure saddle
8	defender 2 ahead, same row, on a goal row	3×2	degenerate (row ties)
4	defender 2 ahead, carrier off the goal row	3×3	wider mix

The **four 2×2-support templates are all verified matching pennies** — the carrier's best reply flips between the two defender columns and vice versa — covering **68 of 94** states: carrier {climb, advance} against defender {cover a lane, hold the forward cell}. All 94 share one value across every equilibrium; 64 have a unique one.



One policy fan per matching-pennies template. Each covers a class of mixed states related by carrier-frame geometry; p⋆, q⋆ are the equilibrium weights on the first support action. The carrier randomizes between two scoring lanes, the defender between covering them.

Why the mix is unavoidable. A mixed stage game needs all three of: a stochastic (½–½) resolution order; a goal mouth wider than one cell, so the carrier has two scoring lanes; and a defender close enough to contest the forward cell but unable to cover both lanes in one move. The carrier climbs or drops, the defender guesses which — matching pennies. Drop any one clause — a single goal cell, deterministic resolution, or the coin-flip tie-break — and every stage game has a pure saddle. The full argument and one stage matrix per template are in `docs/templates.md`. For the single goal cell the theorem is partly proved (`docs/proof.md`): the defender has a closed-form optimal strategy — guard the one cell — and every stage game is dominance-solvable, machine-checked for all boards up to 11×5.

SHALLOW MIXES, BUT THEY COMPOUND

Every one of the 94 no-pure-saddle stage games sits within `0.07` of a pure saddle (`minimax − maximin`, median `0.029`) — the equilibria are shallow, and the exact mixing weights are numerically delicate. What is robust: 68 have a 2×2 support and all 68 are structurally matching pennies, and the *value of mixing* — `V(hybrid) − V(pure maximin)` — is `+0.13` on average at those states. It compounds along a path: at the centred kickoff a pure-strategy player secures only a draw (`0.000`) where mixing is worth `+0.15` (`scripts/mixing.py`, `docs/mixing.md`).

The reward objective does not move the mixed region

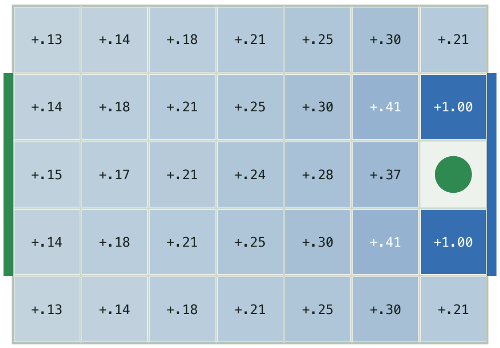
The default game rewards a *win* — first goal ends it, value `P(win) − P(loss)`. A second objective, `scoring="rate"` (`scripts/reward.py`), keeps playing: a goal scores ±1 and restarts with the ball to the conceding team, so the value is the expected discounted *goal difference* and `γ < 1` is load-bearing. On the 7×5 random-order board:

SAME DYNAMICS, TWO REWARD OBJECTIVES — RANDOM MOVE ORDER, DISCOUNT 0.9

OBJECTIVE	NO-SADDLE STATES	VALUE RANGE	V(KICKOFF)
win — P(win) − P(loss)	94	[−1.00, +1.00]	+0.150
rate — expected goal difference	94 — the same 94	[−0.88, +0.88]	+0.132

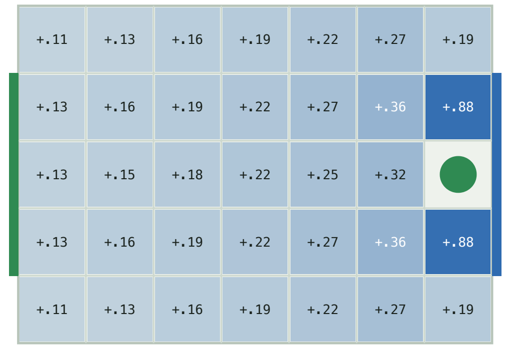
The no-pure-saddle set is *identical* — **0** states enter or leave. A mixed Nash equilibrium is an equilibrium of a single stage game `M(s)`; changing how the horizon is scored rescales `M(s)` through `V(s′)` but does not cross the `maximin = minimax` boundary. The value *range* compresses (`±1 → ±0.88`): under `rate` a lost position is not a cliff — conceding restarts play with possession, a floor under every state. Details in `docs/reward.md`.

scoring = win (P(win) - P(loss))



V* by player 0 position (blue = player 0 ahead)

scoring = rate (expected goal difference)



V* by player 0 position (blue = player 0 ahead)

Same shape, compressed range. V* by carrier position under the two objectives. `rate` pulls the goal-adjacent +1 cells down to +0.88 — the concede-and-restart floor — without changing where the value gradient runs.

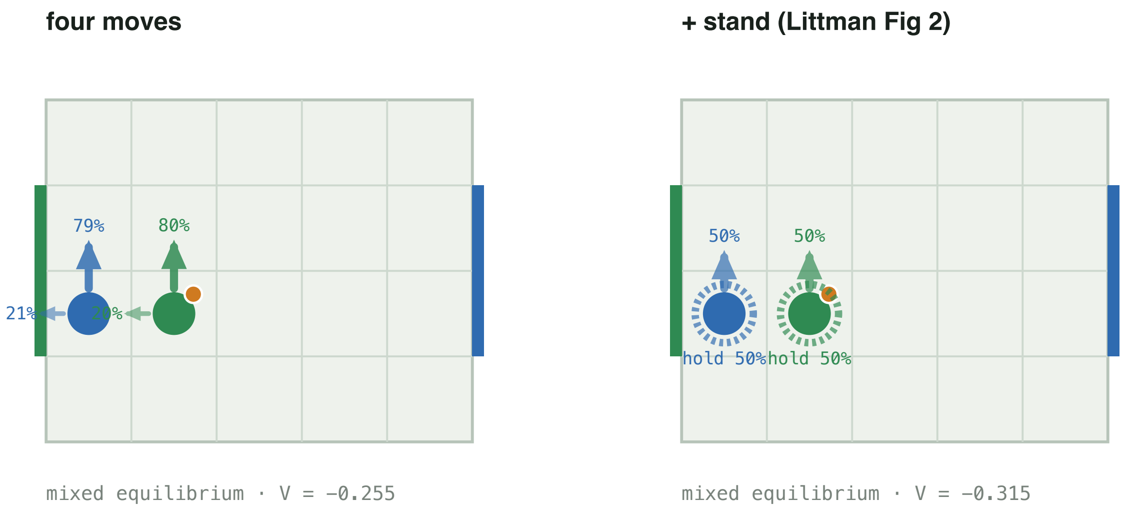
Littman's fifth action

Littman's 1994 soccer game has a fifth action, `stand`, and his Figure 2 — a carrier pinned against its own goal by an adjacent defender — is the textbook "soccer needs mixing" example: the carrier must randomize between standing and moving. Adding `stand` to the random-order game (`n_actions=5`, `scripts/littman.py`) on Littman's 5×4 board:

LITTMAN'S BOARD (5×4, 2-CELL GOALS), RANDOM MOVE ORDER, DISCOUNT 0.9

ACTION SET	NO-SADDLE STATES	STAND IN THE MIXED SUPPORT	V(KICKOFF)
N, S, E, W	56	0	+0.147
N, S, E, W, stand	56	48	+0.172

The fifth action does *not* move where mixing happens — 48 of the 56 no-pure-saddle states are shared, 8 swap in and 8 out at the margin — but it changes *what* the mix is: the carrier now hedges with `stand` in 48 of 56 states rather than with `retreat`, and every player is slightly better for the option (`V(kickoff) +0.147 → +0.172`). The Figure 2 state is a clean matching-pennies matrix with equilibrium (`½ climb, ½ hold`). The deterministic game keeps a pure saddle at every state with five actions too.



Littman's Figure 2. Same state, four moves (left) vs. five (right). The dashed ring is `stand`. Adding it turns the carrier's climb/retreat hedge into climb/hold — Littman's mixed policy.

RQ4 — Numerical robustness

The value is three numbers

`value_bracket` returns what the row player guarantees ($\min_k (pM)_k$), gets ($p^\top Mq$), and can reach ($\max_i (Mq)_i$). The true value is bracketed, so the width certifies the LP error.

> FINDING
>
> With `scipy`'s HiGHS the three agree to `1.1e−16` per stage game, `8.7e−10` accumulated. The concern is real for older vertex/simplex solvers — a non-issue here.

Discounting and tolerance

	DETERMINISTIC	RANDOM (7×5)
mixed states, γ = 0.5 → 0.9 → 0.995	0 → 0 → 0	122 → 94 → 102
classification split, rel_tol 1e−12 → 1e−3	stable	604 / 1682 / 94 (stable)
classification, rel_tol 1e−2 / 1e−1	—	80 / 0 mixed
fixed 0.1-rounding flips a saddle	0	62

Discounting mildly shifts which states mix; the classification is otherwise robust across nine orders of magnitude of tolerance, breaking only as the tolerance approaches the real `minimax − maximin` gaps. Fixed 0.1-rounding — proposed to force indifference — misclassifies 62 states, exactly the small-entry mixed region.

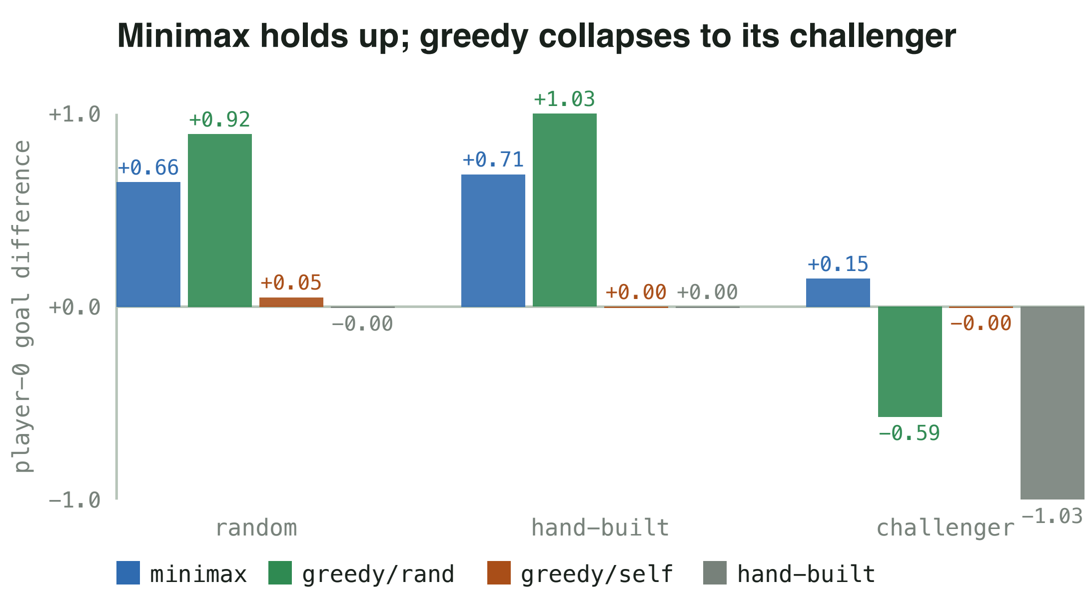
Secondary validation — the policy plays correctly

Littman's Table 3: minimax exploits *and* survives

Littman's central experiment trains four policies and scores each against a random opponent, a hand-built one, and a **challenger** trained to beat it — his minimax policies held ~35–40% of games against their challenger, his Q-learning (deterministic) policies won **0%**. Reproduced here with the *exact* solver on his 5×4 / 5-action / random-order / goal-reset board (`scripts/tournament.py`, score = player-0 expected discounted goal difference):

LITTMAN'S BOARD, Γ = 0.9 — PLAYER-0 GOAL DIFFERENCE FROM THE KICKOFF

PLAYER-0 POLICY	VS. RANDOM	VS. HAND-BUILT	VS. ITS CHALLENGER	ROBUSTNESS GAP
minimax (Nash)	+0.67	+0.71	+0.15	+0.52
greedy vs. random	+0.92	+1.03	−0.59	+1.51
greedy self-play	+0.05	0.00	−0.00	+0.05
hand-built (scripted)	−0.00	0.00	−1.03	+1.03



Only minimax is in the top-left. greedy/rand is the strongest policy against weak opponents and the weakest against a challenger — the rock-paper-scissors trap. minimax presses its advantage (+0.67) and still wins (+0.15) against the worst case, because the Nash policy has no exploitable row.

The mechanism is exactly Littman's "every deterministic offense has a perfect defense": the mixed states are matching-pennies stage games, a greedy policy commits to one row of each, and the challenger plays the column that beats it. The minimax policy randomizes with the weights that make the challenger indifferent. Full table (including the `P(win) − P(loss)` view) in `docs/tournament.md`.

Other checks

- Duality gap `V0_br(s_0) + V1_br(s_0) < 1e-9` for every move order — no opponent beats the game value.
- Nash vs. Nash reproduces the value: forced draws in the deterministic and coin-flip games; `+0.162 ± 0.002` empirical discounted return over 5 seeds of 3000 games (vs. `+0.164` computed) in the random game.
- A best response to one assumed opponent is fragile: the Part 2 policy has exploitability `0.43` — worse than random — which is the A10 competition's warning about non-equilibrium submissions.
- A10 networks over 5 seeds (`experiments/a10_competition_seeds.csv`): the Part 2 imitation net plays optimally at `0.990 ± 0.000` of states and wins Q8 every time; competition Network Second is exact, but Network First sits at a `0.19 ± 0.02` exploitability from this kickoff that does not wash out with re-seeding.

Beyond zero-sum, and open questions

- **General-sum — extension point.** `markov_game.py` solves 2-player general-sum Markov games by enumerating *all* stage equilibria (`support_enum.py`) and selecting one — the notes' "largest sum of values", a no-op for zero-sum but decisive for Battle of the Sexes, where it avoids the mixed equilibrium whose value is below either pure one. Pure-first throughout. The soccer game is zero-sum, so this does not touch the main result.
- **Function approximation — partial.** A network fit to the stage matrices (`nash_dqn.py`, 5→96→96→16 with biases, 600 epochs) trails the exact solver, over **5 seeds** (`experiments/nash_dqn_seeds.csv`):

	EXACT HYBRID NASH-Q	NEURAL NASH-Q (MEAN ± SD)
max |V − V_exact|	0	0.55 ± 0.02
mean |V − V_exact|	0	0.13 ± 0.00
action agreement	100%	43% ± 2%
pure/mixed classification	100%	67% ± 2%
exploitability	<1e-9	0.44 ± 0.05
convergence (epochs to plateau, of 600)	exact	469 ± 31; 3/5 still creeping
runtime	9 sweeps, 0.3 s	600 epochs, ~35 s

Deterministic game only — `train_nash_dqn` rejects the stochastic variants, so the network never has to represent a genuinely mixed stage game. Even so it trails: it fits the ~1 000 zero-value states easily (small *mean* error) but misses the γ^k bands and contested regions — worst-state error `0.55`, policy about as exploitable as random play, mis-calls *whether* a state needs mixing a third of the time. Keep the exact solver as ground truth.

- **Four research questions** — the single-cell theorem's last step, whether the goal-width dichotomy is a known result, the move-order-vs-randomness distinction, and why freeze-then-iterate thrashes — are stated and answered as far as the evidence allows in `docs/discussion.md`.

Limitations

- The A10 page redacts the ID-specific start position and goal rows; this repo uses the standard Littman geometry (configurable). Across the phase-diagram sweep every one-cell-goal board has 0 mixed states and every wider-goal board has some; the exact `3.95%` is the 7×5 / 3-cell-goal case.
- Several collision sub-cases (carrier vs. stationary opponent, bump = steal, swap possession) are *interpreted* from the two stated A10 rules, not quoted. If course staff intended otherwise, Claim A must be re-measured. See `docs/assumptions.md`.
- **Claim A/B** (a pure equilibrium of the deterministic game) is established constructively — the explicit pure memoryless attractor/safety profile. **Claim C** (the theoretical game necessarily has one, over all rules and settings) is *not*, for the general goal mouth; for the single goal cell it is partly proved (`docs/proof.md`).
- `random` and `coinflip` are different game definitions, isolating the collision rule and the tie-break respectively — not the A10 game.

`soccer_nash — game.py · matrix_games.py · support_enum.py · nash_q.py · markov_game.py · numerics.py · dominance.py · attractor.py · geometry.py · symmetry.py · tree.py · templates.py · onecell.py · reachability.py · occupancy.py · best_response.py · exploit.py · mlp.py · nash_dqn.py · opponents.py · shaping.py · simulate.py · experiment.py · render.py · viz.py`
`scripts — experiments.py · analyze.py · numerics.py · equilibria.py · geometry_model.py · templates.py · phase_diagram.py · onecell_proof.py · positional.py · benchmark.py · policy_iteration.py · selfplay.py · nash_dqn.py · render.py · gallery.py · littman.py · mixing.py · reward.py · blend.py · occupancy.py · tournament.py · a10_part1.py · a10_part2.py · a10_competition.py`
`docs — design.md · advisor.md · methods.md · discussion.md · assumptions.md · templates.md · proof.md · geometry.md · mechanism.md · littman.md · tournament.md · mixing.md · reward.md · blend.md · occupancy.md · gallery.html`
`experiments — baseline.csv · undiscounted.csv · gamma_sweep.csv · tolerance_sweep.csv · board_sweep.csv · goal_mouth_sweep.csv · one_cell_goal.csv · phase_diagram.csv · nash_dqn_seeds.csv`
`env: 7×5 grid, 2 380 states, zero-sum ±1, γ = 0.9`